Inferenz in C

Teil 4 und 5 — Vorhersage ohne Training

Inhaltsverzeichnis

Datenstrukturen 2
Forward-Pass 2
Teil 4: Laden aus der TXT-Datei 3
Teil 5: Initialisierung aus dem Header 3
Ergebnisse 4
Bauen und testen 4

Teil 4 und 5 der Angabe bauen das Netz in C nach, aber nur den Forward-Pass und die Vorhersage, ohne Training. Die Gewichte kommen in Teil 4 aus der TXT-Datei, in Teil 5 aus dem C-Header. Beide Teile verwenden dieselben Strukturen und Funktionen; der Code liegt im Ordner `c/` des Repos.

```
flowchart LR
    T["xor.txt<br>seven_segment.txt"] --> L["NetworkLoadTxt<br>(Teil 4)"]
                                    H["xor.h<br>seven_segment.h"]         -->
I["XOR_Init<br>SEVEN_SEGMENT_Init<br>(Teil 5)"]
    L --> N["Network"]
    I --> N
    N --> P["NetworkPredict"]
```

Datei	Inhalt
`neural_network.h`, `neural_network.c`	Strukturen `Neuron`, `Layer`, `Network`; Laden aus TXT, Forward-Pass, Vorhersage, Freigeben
`predict_txt.c`	Teil 4: lädt ein Netz aus einer TXT-Datei und testet es an seinen Daten
`predict_header.c`	Teil 5: baut beide Netze aus den Headern und testet sie
`evaluation.h`, `evaluation.c`	Trainingsdaten von XOR und 7-Segment-Decoder, Ausgabe der Tabelle
`selftest.c`	Selbsttest, siehe unten
`Makefile`	Bauen und Testen mit `make`

Datenstrukturen

```
typedef struct
{
    // One weight per input
    const double *weights;
    // Bias term b
    double bias;
    // Number of inputs, i.e. neurons in the previous layer
    int inputCount;
    // Output a of the last forward pass
    double output;
} Neuron;

typedef struct
{
    Neuron *neurons;
    int neuronCount;
    int inputCount;
} Layer;

typedef struct
{
    Layer *layers;
    int layerCount;
    int inputSize;
} Network;
```

`weights` zeigt in Teil 4 auf Speicher, den `NetworkLoadTxt` reserviert, in Teil 5 direkt auf die konstanten Arrays im Header. So passt dieselbe Struktur für beide Teile, und in Teil 5 wird kein einziges Gewicht kopiert.

Forward-Pass

```
void NetworkForward (Network *network, const double *inputs)
{
    for (int l = 0; l < network->layerCount; l++)
    {
        Layer *layer = &network->layers[l];

         // The first layer reads the network inputs, every other layer the
outputs before it
        const Layer *previous = l > 0 ? &network->layers[l - 1] : NULL;

        for (int n = 0; n < layer->neuronCount; n++)
        {
            Neuron *neuron = &layer->neurons[n];
```

```
            double sum = 0.0;
            for (int i = 0; i < neuron->inputCount; i++)
            {
              double input = previous != NULL ? previous->neurons[i].output :
inputs[i];
                sum += neuron->weights[i] * input;
            }
            neuron->output = Sigmoid(sum + neuron->bias);
        }
    }
}
```

Jeder Layer liest die Outputs, die die Neuronen des vorherigen Layers gespeichert haben; ein Zwischenpuffer ist nicht nötig. `NetworkPredict` ruft den Forward-Pass auf und kopiert die Outputs des letzten Layers in ein Array des Aufrufers. Die Sigmoid-Funktion hat dieselben zwei Zweige wie in Python und C#.

Teil 4: Laden aus der TXT-Datei

Weil vor jedem Wert ein Schlüsselwort steht, liest ein einziger `fscanf`-Aufruf den Kopf der Datei:

```
int fields = fscanf(file, " insy-nn %d activation %31s input_size %d layers
%d", &version,
                    activation, &network->inputSize, &layerCount);
```

Danach folgen pro Layer `layer <n> neurons <anzahl> inputs <anzahl>` und pro Neuron `neuron <n> bias <wert> weights <werte>`. Das Programm prüft Version, Aktivierung, die Nummerierung und ob jeder Layer so viele Inputs hat wie der vorherige Neuronen. Bei einem Fehler meldet es den Grund, gibt den schon reservierten Speicher frei und liefert `-1`.

Teil 5: Initialisierung aus dem Header

Der Header enthält neben den Konstanten die Funktion `<PRÄFIX>_Init`. Python und C# erzeugen sie mit `dump_initfile` bzw. `DumpInitFile`, byte-gleich in beiden Sprachen. Das ganze Programm für den 7-Segment-Decoder braucht damit weder Datei noch `malloc`:

```
#include "seven_segment.h"

Network network;
SEVEN_SEGMENT_Init(&network);

double outputs[7];
NetworkPredict(&network, (const double[]){0, 1, 0, 1}, outputs);
```

Ergebnisse

Netz	aus TXT (Teil 4)	aus Header (Teil 5)
XOR	4/4 richtig: 0,0343 / 0,9699 / 0,9708 / 0,0308	4/4 richtig, dieselben Ausgaben
7-Segment-Decoder	10/10 Ziffern	10/10 Ziffern

Die Ausgaben sind dieselben wie in Python und C# (siehe Ergebnisse). Getestet wurde mit den Modellen aus beiden Sprachen.

Der Selbsttest prüft zusätzlich:

- Sigmoid bei extremen Werten und einen Forward-Pass gegen die von Hand ausgerechnete Formel,
- dass eine falsche Datei (der Header statt der TXT-Datei) sauber abgelehnt wird,
- dass das Netz aus der TXT-Datei und das aus dem Header für **jeden** möglichen 0/1-Input bitgleiche Ausgaben liefern. `fscanf` liest also jede Zahl exakt so ein, wie der Compiler sie aus dem Header übernimmt.

Bauen und testen

Gebaut wird mit GCC 16.2 und `make` aus w64devkit, einer portablen Werkzeugsammlung, die ohne Installation läuft. Im Ordner `c/`:

```
make test
make MODELS=../models/csharp test
```

Mit `-Wall -Wextra -Wpedantic -Wconversion -Wshadow` übersetzt der Code ohne eine einzige Warnung.